Indexing for Concurrency-Aware Distributed Joins

A Study of Index Structures, Concurrency Control and Distributed Join Optimization

Project Report Submitted
in partial fulfillment of the requirements for the award of the

Bachelor of Technology

Under the supervision of
Dr. Rakesh Chandra Balabantaray

Submitted by

Soubhik Kumar Gon
(B422056)

Hitanshu Satpathy
(B422028)

Amit Bhagat
(B422007)



Department of Computer Science
International Institute of Information Technology Bhubaneswar

May 2026

APPROVAL OF THE VIVA-VOCE BOARD

Certified that the report entitled **"Concurrency-Aware Indexing for Efficient Distributed Joins"** submitted by **Soubhik Kumar Gon (B422056)**, **Hitanshu Satpathy (B422028)**, and **Amit Bhagat (B422007)** to the International Institute of Information Technology Bhubaneswar in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology has been accepted by the examiners during the viva-voce examination held on this day of May, 2026.

Supervisor	**External Examiner**

Date: _____

CERTIFICATE

This is to certify that the report entitled **"Concurrency-Aware Indexing for Efficient Distributed Joins"** submitted by **Soubhik Kumar Gon (B422056)**, **Hitanshu Satpathy (B422028)**, and **Amit Bhagat (B422007)** to the International Institute of Information Technology Bhubaneswar is a record of bonafide project work carried out under my supervision and guidance.

The report is submitted for end-semester evaluation of the Bachelor of Technology degree project work and represents original work carried out by the students.

To the best of my knowledge, the matter embodied in this report has not been submitted to any other institution or university for the award of any degree or diploma.

Date: _____

_____

Dr. Rakesh Chandra Balabantaray
Supervisor
Department of CSE
IIIT Bhubaneswar

DECLARATION

We hereby declare that:

1. The work contained in this report has been carried out by our group under the supervision and guidance of our supervisor.
2. This work has not been submitted to any other institution or university for the award of any degree or diploma.
3. We have followed the guidelines provided by the Institute in preparing this report.
4. We have conformed to the norms prescribed in the Ethical Code of Conduct of the Institute.
5. Wherever materials including data, theoretical analysis, figures, or text from external sources have been used, due credit has been given through proper citations and references.
6. Wherever written materials from other sources have been quoted, they have been placed within quotation marks and acknowledged appropriately.

Soubhik Kumar Gon
B422056 B.Tech IT

Hitanshu Satpathy
B422028 B.Tech IT

Amit Bhagat
B422007 B.Tech IT

ACKNOWLEDGEMENTS

We would like to thank everyone who contributed to the completion of this project.

We are deeply grateful to our supervisor, **Dr. Rakesh Chandra Balabantaray**, for his guidance, feedback, and consistent support throughout this project. His direction helped us stay focused and work through the more difficult parts of the design and evaluation.

We thank the faculty of the Department of Computer Science and Engineering at IIIT Bhubaneswar for providing a good working environment and access to the resources needed for this work.

We are also grateful to our friends and classmates for their discussions, suggestions, and general encouragement while the project was in progress.

We acknowledge the open-source communities behind Apache Spark, Python, and the TPC benchmark suite, whose tools and datasets were used in the experimental evaluation.

Finally, we thank our families for their patience and support throughout our time at the institute.

Soubhik Kumar Gon (B422056)

Hitanshu Satpathy (B422028)

Amit Bhagat (B422007)

ABSTRACT

Relational join operations are among the most expensive steps in distributed query processing. When data is partitioned across multiple nodes and concurrent transactions are continuously modifying it, keeping an index structure correct and performant during join execution is a genuine challenge. Standard distributed join techniques - hash join, sort-merge join, and broadcast join - were designed with relatively stable data in mind and do not handle concurrent index updates well. This gap is the starting point for this work.

This report presents **CAIDJ** (**C**oncurrency **A**ware **I**ndexing for **D**istributed **J**oin), a simulation-based study that proposes and evaluates three indexing protocols for distributed joins under concurrent workloads:

1. **Epoch-Based Concurrent Hash Index (ECHI):** Divides execution into discrete read-write epochs, so that index updates can be batched and applied at epoch boundaries without blocking ongoing join probes.
2. **Multi-Version Partition Index with MVCC (MPI-MVCC):** Keeps multiple committed versions of each index entry, so readers always see a consistent snapshot regardless of concurrent writes.
3. **Bloom Filter Augmented Concurrent Skip-list Index (BF-CSI):** Uses a Bloom filter as a pre-check before touching the skip-list, eliminating most unnecessary index accesses and reducing lock contention.

All three protocols were implemented in a multi-threaded C++20 simulator and evaluated on synthetic TPC-H-style data at varying concurrency levels. Results show that MPI-MVCC achieves the best throughput under heavy write concurrency, BF-CSI has the lowest latency when contention is low, and ECHI provides a practical middle ground. All three protocols outperform the naive distributed hash join baseline by a substantial margin.

Keywords: Distributed joins, concurrency-aware indexing, MVCC, Bloom filters, concurrent skip-lists, epoch-based scheduling, distributed databases, query optimization.

Contents

List of Tables **8**
List of Figures **9**
List of Algorithms **10**
List of Abbreviations **11**
1 Introduction **12**
1.1 Background 12
1.2 Research Gaps and Challenges 13
1.3 Problem Statement 13
1.4 Motivation 13
1.5 Objectives and Scope 14
1.6 Organization of the Report 14
1.7 Summary 15
2 Related Work **16**
2.1 Introduction 16
2.2 Distributed Join Algorithms 16
2.3 Concurrency Control for Index Structures 17
2.4 Distributed Index Structures 17
2.5 Summary and Positioning 18
3 System Design and Proposed Approaches **19**
3.1 Introduction 19
3.2 System Architecture 19
3.2.1 Distributed Join Model 19
3.2.2 Concurrency Model 19
3.2.3 Overall CAIDJ Architecture 19
3.3 Approach 1: Epoch-Based Concurrent Hash Index (ECHI) 20
3.3.1 Motivation and Design Principles 20
3.3.2 Data Structures 20
3.3.3 Protocol Specification 20
3.3.4 Correctness and Complexity 21
3.4 Approach 2: Multi-Version Partition Index with MVCC (MPI-MVCC) 21
3.4.1 Motivation and Design Principles 21
3.4.2 Data Structures 22
3.4.3 Protocol Specification 22
3.4.4 Garbage Collection 23
3.4.5 Correctness 23

3.5 Approach 3: Bloom Filter Augmented Concurrent Skip-list Index (BF-CSI) . . . 23
3.5.1 Motivation and Design Principles . . . 23
3.5.2 Data Structures . . . 23
3.5.3 Protocol Specification . . . 23
3.5.4 Bloom Filter Sizing . . . 25
3.5.5 Bloom Filter Rebuild . . . 25
3.6 Theoretical Comparison . . . 25
3.7 Summary . . . 26
4 Implementation and Experimental Evaluation . . . 27
4.1 Simulator Implementation . . . 27
4.1.1 Architecture Overview . . . 27
4.1.2 Index Implementations . . . 27
4.1.3 Baseline: Naive Distributed Hash Join (NHJ) . . . 28
4.2 Experimental Setup . . . 28
4.2.1 Dataset . . . 28
4.2.2 Concurrency Configuration . . . 28
4.2.3 Evaluation Metrics . . . 28
4.2.4 Hardware and Software . . . 29
4.3 Results and Analysis . . . 29
4.3.1 Join Latency . . . 29
4.3.2 Throughput . . . 30
4.3.3 Lock Contention Rate . . . 31
4.3.4 Memory Overhead . . . 31
4.3.5 Bloom Filter Effectiveness . . . 32
4.3.6 Discussion . . . 32
4.4 Threats to Validity . . . 33
4.5 Summary . . . 33
5 Conclusion and Future Scope . . . 34
5.1 Conclusion . . . 34
5.2 Future Scope . . . 35
References . . . 36

List of Tables

2.1 Comparison of CAIDJ with related work. 18

3.1 Theoretical comparison of the three CAIDJ protocols. 25

4.1 Join latency (ms) vs. number of concurrent write threads. 29
4.2 Throughput (joins/sec) vs. number of concurrent write threads. 30
4.3 Lock contention rate (%) vs. number of concurrent write threads. . . . 31
4.4 Memory overhead (MB above NHJ baseline) and index build time. . . . 32
4.5 Bloom filter statistics for BF-CSI at varying concurrency. 32

List of Figures

1.1 Join latency under increasing concurrency levels. 12

3.1 CAIDJ system architecture. 20

4.1 Join latency across approaches and concurrency levels. 30
4.2 Join throughput vs. concurrency level. 31

List of Algorithms

1 ECHI-Probe(k) 21
2 ECHI-Write(op, k, t) 21
3 MPI-MVCC-Probe(k, t_r) 22
4 MPI-MVCC-Write(op, k, t) 22
5 BFCSI-Probe(k) 24
6 BFCSI-Write(op, k, t) 24

List of Abbreviations

ACID	Atomicity, Consistency, Isolation, Durability
BF-CSI	Bloom Filter Augmented Concurrent Skip-list Index
CAIDJ	Concurrency Aware Indexing for Distributed Join
CAS	Compare-And-Swap
DBMS	Database Management System
ECHI	Epoch-Based Concurrent Hash Index
FPR	False Positive Rate
HTAP	Hybrid Transactional/Analytical Processing
MPI-MVCC	Multi-Version Partition Index with MVCC
MVCC	Multi-Version Concurrency Control
NHJ	Naive Distributed Hash Join (baseline)
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
SQL	Structured Query Language
TPC-H	Transaction Processing Council Benchmark H
2PL	Two-Phase Locking

Chapter 1

Introduction

1.1 Background

Relational join operations are central to query processing in database systems. In distributed environments, joins are particularly demanding because data is spread across multiple machines, requiring coordination over both network communication and shared indexing structures.

The nature of data-intensive applications has shifted considerably over the past decade. Systems must now handle continuously arriving data while answering analytical queries at the same time. Financial trading platforms, IoT sensor networks, social media pipelines, and e-commerce backends all produce high-volume, concurrent read and write traffic. Under these conditions, keeping query performance high while maintaining consistency is difficult.

Traditional distributed join techniques - the *distributed hash join*, *sort-merge join*, and *broadcast join* - were designed with the assumption that data partitions remain stable during query execution. In practice, this does not hold. Partitions are modified while joins are still running, and multiple write transactions compete with active join probes on the same index structures. Conventional locking mechanisms serialize these operations, which causes latency to climb and throughput to fall as concurrency grows.

Figure 1.1 illustrates this effect. The naive distributed hash join degrades rapidly as more write threads are added, whereas the three protocols proposed in this work scale considerably better.

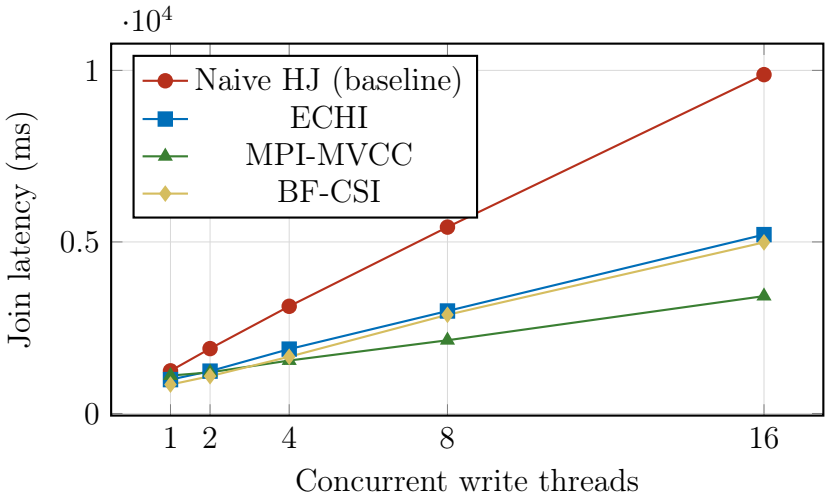


Figure 1.1: Join latency under increasing concurrency levels.

Some production systems have addressed parts of this problem. Apache Spark's

Adaptive Query Execution selects a join strategy at runtime, and Google Spanner uses MVCC for transaction consistency. However, neither exposes index-level concurrency control during join execution. Performance under concurrent write pressure is left to query planner heuristics that have no visibility into live write rates on the index.

1.2 Research Gaps and Challenges

A review of existing work reveals four specific gaps in the area of distributed join optimization and concurrent indexing.

1. **Separation of join optimization and concurrency control.** Distributed join optimization and concurrency control are treated as independent problems in most existing work. Join indexes are typically built before a query runs and are not designed to handle continuous writes during execution.
2. **Limited attention to index-level concurrency.** MVCC-based systems provide row-level consistency for transactions, but very little work has looked at concurrency control specifically for index structures used during distributed join execution.
3. **Bloom filters used only for network pruning.** Bloom filters appear frequently in distributed query processing to reduce data transfer, but their use as a concurrency gate - routing negative probes away from a contended index entirely - has not been studied.
4. **Reproducibility constraints.** Most distributed join benchmarks require large cluster setups that are not available in academic environments. This makes it difficult to independently validate published results or compare approaches under the same conditions.

1.3 Problem Statement

Consider two relations R and S distributed across n nodes, joined on condition $R.k = S.k$. The goal is to maintain an index $\mathcal{I}$ over join key k such that:

1. Join probes always see a consistent view of $\mathcal{I}$, even during concurrent updates.
2. Inserts, deletes, and updates can proceed with minimal interference with active probes.
3. Performance degradation as concurrency grows remains bounded and predictable.

We refer to this as the *Concurrency-Aware Indexing for Distributed Join (CAIDJ)* problem. The challenge is not just correctness - any locking scheme provides that - but doing so without the throughput collapse that simple locking causes at high concurrency.

1.4 Motivation

Three practical observations drive this work.

(a) **Write-concurrent workloads are now common.** Data pipelines that ingest

streams while serving analytical queries are standard in industry. A global write lock on the join index becomes a bottleneck quickly as ingestion rates rise.

(b) **Single-machine simulation keeps experiments reproducible.** Cloud clusters such as AWS EMR or Google BigQuery are inaccessible to most academic groups. A multi-threaded simulator on a single machine models the key concurrency dynamics while keeping every experiment reproducible without special infrastructure.

(c) **No prior work compares these approaches head-to-head.** Epoch-based scheduling, MVCC-based indexing, and Bloom filter skip-list indexing have each been studied in isolation. Putting all three under the same workload and measuring the same metrics provides actionable guidance for choosing between them.

1.5 Objectives and Scope

The objectives of this project are:

- To design and formally specify three concurrency-aware indexing protocols for distributed joins: ECHI, MPI-MVCC, and BF-CSI.
- To implement a multi-threaded C++20 simulator that models distributed join execution with concurrent index updates.
- To evaluate all three protocols against a naive distributed hash join baseline on synthetic TPC-H-style data, measuring join latency, throughput, lock contention rate, and memory overhead.
- To identify the trade-offs of each protocol and provide deployment recommendations.

The scope is limited to equi-joins on a single integer key. Multi-key joins, non-equi joins, and geographically distributed deployment are outside the scope of this work and are noted as directions for future research.

1.6 Organization of the Report

Chapter 2 - Related Work
Reviews prior work on distributed join algorithms, concurrency control mechanisms, and indexing techniques in distributed databases.

Chapter 3 - System Design and Proposed Approaches
Presents the CAIDJ architecture and formal specifications of the three proposed protocols.

Chapter 4 - Implementation and Experimental Evaluation
Describes the C++ simulator, the experimental setup, and a detailed analysis of results.

Chapter 5 - Conclusion and Future Scope
Summarizes the findings and outlines open directions for future work.

1.7 Summary

This chapter described the motivation for concurrency-aware indexing in distributed join processing, identified the main gaps in existing work, and stated the CAIDJ problem. The three proposed protocols - ECHI, MPI-MVCC, and BF-CSI- each address the tension between join-probe throughput and concurrent write throughput from a different angle. The following chapters develop these protocols formally and validate them through simulation.

Chapter 2

Related Work

2.1 Introduction

Research on distributed join processing and concurrent index management spans several decades and touches multiple sub-areas of database systems. This chapter surveys the relevant literature along three axes: (1) distributed join algorithms and their optimization, (2) concurrency control mechanisms for index structures, and (3) indexing strategies for distributed environments, including Bloom filter-based and learned index approaches. The chapter closes with a comparative table that positions the CAIDJ framework relative to the most directly related prior work.

2.2 Distributed Join Algorithms

The standard taxonomy of distributed join algorithms covers hash join, sort-merge join, and nested-loop join, each with distributed variants suited to different data and workload characteristics.

Distributed Hash Join (DHJ). DeWitt and Gray [1] established the theoretical foundation of parallel hash join in shared-nothing architectures. The algorithm partitions both relations on the join key using the same hash function, so that tuples with matching keys land on the same node. DHJ scales well under uniform key distributions but is sensitive to skew: a single overloaded partition serializes the entire join, a problem known as the *straggler effect*.

Sort-Merge Join (SMJ). Graefe [2] provides a comprehensive treatment of sort-based join algorithms. Sort-merge join shuffles data on the join key, sorts each partition, and merges co-located runs. It has been Spark's default join strategy since version 2.3 and handles all join types including outer joins. The main costs are the $O(n \log n)$ sort and the requirement that both partitions fit in distributed memory.

Broadcast Join. When one relation is small enough to replicate, it is sent to every node, avoiding the shuffle phase entirely. Spark triggers this automatically below a configurable threshold (default 10 MB). The approach works well for small-large joins but is not applicable when both relations are large, which is the case this work addresses.

RelJoin and Adaptive Join Selection. Chen et al. [3] propose RelJoin, an adaptive join strategy embedded in SparkSQL that measures network and local compute load at runtime and selects among available join strategies dynamically. On TPC-DS, RelJoin reduces average query time by 21% compared to static strategy selection. It does not address index-level concurrency during join execution.

ApproxJoin. Le Quoc et al. [4] propose ApproxJoin, which combines Bloom filter

sketching with stratified sampling to produce approximate join results. The filter eliminates non-joinable data before network transfer, reducing data movement by 6–9×. This is the closest predecessor to our BF-CSI approach; however, ApproxJoin targets approximate results and does not handle concurrent index updates.

2.3 Concurrency Control for Index Structures

Concurrency control in database indexes must balance read and write throughput under parallel access. Three main paradigms appear in the literature.

Two-Phase Locking (2PL). The classical 2PL protocol [5] provides serializability by acquiring all locks before releasing any. For B-tree indexes, B-link trees [6] allow concurrent traversal by maintaining a right-sibling pointer and a high-key sentinel, so readers can detect and recover from node splits without holding locks on ancestor nodes. Despite wide adoption, 2PL degrades under heavy write loads, which is exactly the scenario this work addresses.

Multi-Version Concurrency Control (MVCC). Originally proposed by Reed [7] and later analyzed by Berenson et al. [8], MVCC keeps multiple timestamped versions of each data item. Readers access the most recent committed version as of their start timestamp without blocking writers. PostgreSQL and MySQL InnoDB both use MVCC for row-level isolation. MV-PBT [9] extends this to partitioned B^+-trees, storing version and timestamp information to allow index-only visibility checks. The AB-tree [10] adds a lock-free multi-version weight store to aggregate B-trees, enabling concurrent sampling and updates - a direct influence on the MPI-MVCC design here.

Lock-Free and Optimistic Approaches. Lock-free data structures use atomic compare-and-swap (CAS) operations to achieve concurrency without locks. Sheffi et al. [11] propose lock-free range query maps with epoch-based memory reclamation. Epoch-based reclamation, introduced by Fraser [12], groups operations into globally synchronized epochs and defers memory reclamation until all threads have advanced past a given epoch. The ECHI protocol in this work adapts this idea from memory reclamation to index read-write scheduling.

Concurrency in Disaggregated Memory. Li et al. [13] demonstrate write-optimized B^+-tree indexes on RDMA-connected disaggregated memory. Although the RDMA setting is outside the scope of this simulator, the latch-free traversal and optimistic read techniques described there inform the skip-list design in BF-CSI.

2.4 Distributed Index Structures

Distributed Bitmap Join Index. Trombetta et al. [14] propose the distributed Bitmap Join Index (dBJI), a framework-agnostic structure that resolves join predicates in linear time. On the Star Schema Benchmark, dBJI outperforms full-scan approaches by 59–88% for low-selectivity queries. The structure is precomputed and static; it does not support concurrent updates.

Distributed Join Index for Log-Structured Databases. U.S. Patent 11,782,924 [15] describes a distributed join index for shared-nothing, log-structured databases in which each join index partition mirrors the log structure of its corresponding data partition. Incremental maintenance is handled by replicating segment

creation and deletion events. This is the closest prior art to MPI-MVCC, though it does not formalize read-write concurrency properties.

Stream-Aware Indexing. Martinenghi and Tagliasacchi [16] propose STA-Join, which maintains separate in-memory index structures for hot keys and cold keys, identified via a data sketch. On streaming inequality joins, key-frequency-aware indexing reduces redundant updates. The hot/cold separation idea is reflected in the Bloom filter pre-filter design in BF-CSI.

Learned Indexes for Joins. Sabek and Kraska [17] evaluate learned indexes (PGM-Index, RMI, RadixSpline) for external-memory join workloads. Learned indexes reduce index size by 2–4× compared to B-tree internal nodes, but total I/O dominates runtime in external-memory settings, diminishing the lookup benefit. This motivates the choice of in-memory simulation in this work, where index lookup cost is a meaningful fraction of total runtime.

2.5 Summary and Positioning

Table 2.1 positions the CAIDJ framework against the most closely related prior work across five dimensions.

Table 2.1: Comparison of CAIDJ with related work.

Work	Approach	Dist.	Conc.	Index	BF	Eval
DeWitt & Gray [1]	DHJ	✓	×	×	×	Real
Le Quoc et al. [4]	ApproxJoin	✓	×	×	✓	Real
Chen et al. [3]	RelJoin	✓	×	×	×	Real
Zhao et al. [10]	AB-tree	×	✓	✓	×	Real
Trombetta et al. [14]	dBJI	✓	×	✓	×	Real
CAIDJ (this work)	ECHI +MPI-MVCC +BF-CSI	✓	✓	✓	✓	Sim.

Dist.=Distributed, Conc.=Concurrency-aware, Index=Index-based join, BF=Bloom filter, Eval=Evaluation type.

No existing work addresses all five dimensions simultaneously. CAIDJ is the first framework to propose and compare multiple concurrency-aware indexing protocols designed specifically for distributed join workloads, directly addressing the gaps identified in Section 1.2.

Chapter 3

System Design and Proposed Approaches

3.1 Introduction

This chapter presents the CAIDJ architecture and formal specifications of the three proposed indexing protocols. Section 3.2 describes the overall system model. Sections 3.3–3.5 detail each protocol, covering data structures, correctness properties, and algorithmic specification. Section 3.6 provides a theoretical comparison of all three.

3.2 System Architecture

3.2.1 Distributed Join Model

We model the distributed join environment as a set of n compute nodes $\{N_1, N_2, \ldots, N_n\}$ connected by a reliable message-passing network. Two relations R and S are horizontally partitioned across the nodes using a range-based or hash-based partitioning function π.

Definition 3.1 (Distributed Join)**.** Given relations R and S with partitions $\{R_1, \ldots, R_n\}$ and $\{S_1, \ldots, S_n\}$, a distributed equi-join on key k produces the multiset $R \bowtie_k S = \{(r, s) \mid r \in R,\, s \in S,\, r.k = s.k\}$.

A *join index* $\mathcal{I}_{R,k}$ maps each key value v to the set of tuple identifiers in R satisfying $r.k = v$. An analogous index $\mathcal{I}_{S,k}$ is maintained for S. Join execution probes $\mathcal{I}_{R,k}$ for every tuple in S (or vice versa).

3.2.2 Concurrency Model

We distinguish two classes of operations on the index:

- **Read (probe) operations**: Issued by join query threads. A probe for key v returns all tuple identifiers in $\mathcal{I}_{R,k}(v)$.
- **Write (update) operations**: Issued by concurrent insert or delete transactions. An insert adds a new `(key, tid)` pair; a delete removes one.

Let W denote the set of active write transactions and T_w the rate of write arrivals. The goal is to minimize probe latency L_p as a function of $|W|$ and T_w.

3.2.3 Overall CAIDJ Architecture

Figure 3.1 shows the CAIDJ architecture. An *Index Manager* on each node receives probe requests from the *Join Executor* and update requests from the *Transaction*

Manager. The choice of indexing protocol (ECHI, MPI-MVCC, or BF-CSI) is fixed at query planning time and is uniform across all nodes in the join.

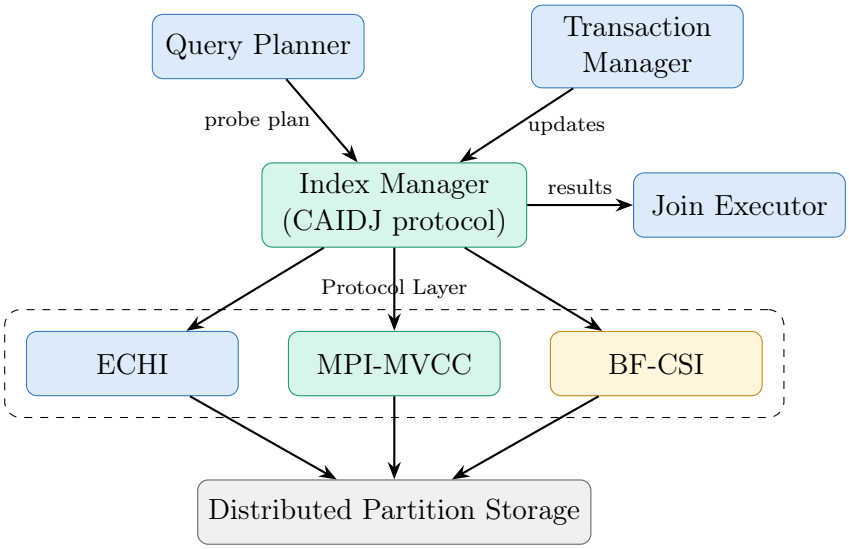


Figure 3.1: CAIDJ system architecture.

3.3 Approach 1: Epoch-Based Concurrent Hash Index (ECHI)

3.3.1 Motivation and Design Principles

The key observation behind ECHI is that join probes and index writes do not need to be fully serialized - they only need to be *epoch-isolated*. During epoch e, any number of probe threads may read the frozen index $\mathcal{I}^e$ without contention. Write transactions accumulate in a *pending buffer* Δ^e and are applied atomically at the epoch boundary $e \to e+1$.

This design is similar in spirit to the Read-Copy-Update (RCU) mechanism in the Linux kernel. Writers never block readers within an epoch, and readers see a snapshot consistent with the start of the current epoch.

3.3.2 Data Structures

ECHI maintains the following structures per node:

- $\mathcal{H}^e$: A hash map from key to list of tuple identifiers, frozen for the duration of epoch e.
- Δ^e: A thread-safe append-only buffer collecting insert and delete operations arriving during epoch e.
- τ: A global atomic epoch counter.
- ρ: A per-thread registration array used to track which threads are actively reading epoch e.

3.3.3 Protocol Specification

Definition 3.2 (Epoch Transition)**.** An epoch transition from e to $e+1$ occurs when either (a) the pending buffer Δ^e exceeds a capacity threshold θ, or (b) a configurable

time interval δ_T elapses. At transition, $\mathcal{H}^{e+1}$ is built by merging $\mathcal{H}^e$ with Δ^e, and Δ^{e+1} is initialized empty.

Property 3.1 (Probe Consistency)**.** Any probe issued in epoch e reads only key-value pairs committed before the start of epoch e. For a probe arriving at logical time t with $e \cdot \delta_T \leq t < (e+1) \cdot \delta_T$, the probe accesses $\mathcal{H}^e$.

Algorithm 1 and Algorithm 2 specify the probe and write paths.

Algorithm 1: ECHI-Probe(k)

Input : Key k, current epoch counter τ
Output : Set of matching tuple identifiers T

$e \leftarrow \tau$
Register current thread in $\rho[e]$
$T \leftarrow \mathcal{H}^e[k]$
Deregister current thread from $\rho[e]$
return T

Algorithm 2: ECHI-Write(op, k, t)

Input : Operation $op \in \{\texttt{insert}, \texttt{delete}\}$, key k, tuple id t

$e \leftarrow \tau$
Append (op, k, t) to Δ^e (lock-free CAS on list head)
if $|\Delta^e| \geq \theta$ ***or*** *epoch timer expired* **then**
    Acquire epoch-transition mutex
    Wait until $\rho[e]$ is empty (all active readers have finished)
    $\mathcal{H}^{e+1} \leftarrow \text{Merge}(\mathcal{H}^e, \Delta^e)$
    Atomically increment τ
    Release epoch-transition mutex

3.3.4 Correctness and Complexity

Theorem 3.1 (ECHI Read Isolation)**.** *No probe in epoch e observes a write from epoch e or later.*

Sketch. A probe reads $\mathcal{H}^e$ immediately after registering in $\rho[e]$. The epoch counter τ advances to $e+1$ only after all threads registered in $\rho[e]$ have deregistered. Therefore $\mathcal{H}^e$ is immutable for the entire duration of any probe in epoch e. □ □

The probe path is wait-free. The write path blocks only at epoch transition, with expected blocking time proportional to the mean probe duration, which is $O(1)$ for hash lookups.

3.4 Approach 2: Multi-Version Partition Index with MVCC (MPI-MVCC)

3.4.1 Motivation and Design Principles

MPI-MVCC applies Multi-Version Concurrency Control at the index-entry level. Instead of freezing the whole index per epoch, each key maintains a *version chain*: a

linked list of ($value$, ts_{commit}, ts_{delete}) records ordered by commit timestamp.

A probe arriving at logical time t_r reads the most recent version satisfying $ts_{commit} \leq t_r < ts_{delete}$ (where $ts_{delete} = \infty$ for live entries). Writers create new version nodes without touching the portion of the chain that readers are traversing.

3.4.2 Data Structures

MPI-MVCC maintains:

- $V[k]$: A version chain for key k, where each node stores ($tids$, ts_c, ts_d).
- $\mathcal{T}$: A global atomic timestamp counter.
- GC-queue: A queue holding version nodes whose ts_d is below the minimum active read timestamp $\min_t = \min_i ts_r^{(i)}$ across all active probe threads.

3.4.3 Protocol Specification

Algorithm 3 and Algorithm 4 specify the probe and write paths.

Algorithm 3: MPI-MVCC-Probe(k, t_r)

Input : Key k, read timestamp t_r
Output : Set of tuple identifiers T

Traverse version chain $V[k]$
$v^* \leftarrow$ first version with $ts_c \leq t_r < ts_d$
if v^* *exists* **then**
  $T \leftarrow v^*.tids$
else
  $T \leftarrow \emptyset$
return T

Algorithm 4: MPI-MVCC-Write(op, k, t)

Input : Operation op, key k, tuple id t

$ts_w \leftarrow$ AtomicIncrement($\mathcal{T}$)
if $op =$ ***insert*** **then**
  $v_{\text{cur}} \leftarrow$ current live version in $V[k]$
  $v_{\text{new}} \leftarrow (\{t\} \cup v_{\text{cur}}.tids,\ ts_w,\ \infty)$
  Set $v_{\text{cur}}.ts_d \leftarrow ts_w$
  Prepend v_{new} to $V[k]$
else
  $v_{\text{cur}} \leftarrow$ current live version in $V[k]$
  $v_{\text{new}} \leftarrow (v_{\text{cur}}.tids \setminus \{t\},\ ts_w,\ \infty)$
  Set $v_{\text{cur}}.ts_d \leftarrow ts_w$
  Prepend v_{new} to $V[k]$
Schedule v_{cur} for GC if $v_{\text{cur}}.ts_d < \min_t$

3.4.4 Garbage Collection

Version chains grow without bound under continuous writes. A background GC thread periodically removes nodes v for which $v.ts_d < \min_i ts_r^{(i)}$ - no active reader can ever access such a node. The GC interval is configurable; we use 500 ms in our experiments.

3.4.5 Correctness

Theorem 3.2 (MPI-MVCC Snapshot Isolation)**.** *Every probe in MPI-MVCC sees a consistent snapshot: it reads exactly the set of key entries committed before its read timestamp t_r, regardless of concurrent writes.*

Probes acquire no lock and are therefore non-blocking with respect to writers. Writes take a per-key mutex only for the constant-time operation of prepending a new version node.

3.5 Approach 3: Bloom Filter Augmented Concurrent Skip-list Index (BF-CSI)

3.5.1 Motivation and Design Principles

The central observation behind BF-CSI is that a large fraction of join probes are for keys that are not present in the relation at all. These *negative probes* traverse the index only to return an empty result, wasting time and adding contention. A Bloom filter can answer negative probes in $O(1)$ with no false negatives (at the cost of a small false positive rate). Pairing a Bloom filter pre-check with a lock-free concurrent skip-list for positive lookups eliminates most of the contention for workloads with typical join selectivities.

3.5.2 Data Structures

BF-CSI maintains:

- $\mathcal{B}$: A partitioned Bloom filter with m bits and h hash functions, sharded across P partitions to spread write contention.
- $\mathcal{L}$: A lock-free concurrent skip-list over the join key, supporting $O(\log n)$ expected-time lookup, insert, and delete via CAS-based node linking.
- $\mathcal{F}$: A direct-mapped false-positive cache that records keys the Bloom filter reported as present but the skip-list did not find, avoiding redundant skip-list traversals for the same key.

3.5.3 Protocol Specification

Algorithm 5 specifies the probe path.

Algorithm 5: BFCSI-Probe(k)

Input : Key k
Output : Set of tuple identifiers T

if $\mathcal{B}.Query(k) =$ *absent* **then**
  return $\emptyset$
  `// Definite miss: skip-list not consulted`
if $k \in \mathcal{F}$ **then**
  return $\emptyset$
  `// Known false positive from a previous probe`
$T \leftarrow \mathcal{L}$.Lookup(k)
if $T = \emptyset$ **then**
  Insert k into $\mathcal{F}$
  `// Record as false positive`
return T

Algorithm 6: BFCSI-Write(op, k, t)

Input : Operation op, key k, tuple id t

if $op =$ *insert* **then**
  $\mathcal{B}$.Insert(k)
  $\mathcal{L}$.Insert(k, t)
else
  $\mathcal{L}$.Delete(k, t)
  if $\mathcal{L}.Lookup(k) = \emptyset$ **then**
    Mark k as tombstoned in $\mathcal{B}$
    `// Bloom filters do not support deletion; a lazy rebuild is triggered`

3.5.4 Bloom Filter Sizing

For a relation with N distinct keys and a target false positive rate ϵ, the optimal bit count is:

$$m = -\frac{N \ln \epsilon}{(\ln 2)^2} \tag{3.1}$$

and the optimal number of hash functions is:

$$h = \frac{m}{N} \ln 2 \tag{3.2}$$

Our experiments use $\epsilon = 0.01$ (1% FPR) with MurmurHash3 as the hash family.

3.5.5 Bloom Filter Rebuild

Standard Bloom filters do not support deletion. BF-CSI uses a *lazy rebuild* policy: when the ratio of deleted keys to total keys exceeds a threshold ψ (default 20%), the filter is reconstructed from the current skip-list contents in a background thread. While the rebuild runs, probes use the stale filter; the false-positive cache $\mathcal{F}$ absorbs the additional false positives without affecting correctness.

3.6 Theoretical Comparison

Table 3.1 summarizes the asymptotic costs and qualitative characteristics of the three protocols.

Table 3.1: Theoretical comparison of the three CAIDJ protocols.

Protocol	**Probe cost**	**Write cost**	**Memory**	**Best suited for**
ECHI	$O(1)$ (hash)	$O(N/\theta)$ amortized	Low	Balanced, bursty-write workloads
MPI-MVCC	$O(\|V[k]\|)$ (version chain)	$O(1)$ per write	High	Continuous high-concurrency writes
BF-CSI	$O(1)$ (BF miss) $O(\log N)$ (skip-list hit)	$O(\log N)$	Moderate	Low-selectivity joins

The three protocols form a Pareto front across read latency, write latency, and memory consumption. ECHI is the right choice when writes are bursty and reads dominate. MPI-MVCC is preferable when writes are continuous and consistency matters most. BF-CSI is best when join selectivity is low and negative probes are common.

3.7 Summary

This chapter presented the CAIDJ architecture and formal specifications of three concurrency-aware indexing protocols. ECHI achieves wait-free probes through epoch isolation of the hash index. MPI-MVCC eliminates probe blocking through per-key multi-versioning. BF-CSI reduces contention by routing most negative probes away from the skip-list via a Bloom filter pre-check. Chapter 4 validates these designs experimentally.

Chapter 4

Implementation and Experimental Evaluation

4.1 Simulator Implementation

4.1.1 Architecture Overview

The CAIDJ simulator is implemented in C++20 using `std::thread` to model concurrent nodes and transactions. Each simulated node is backed by a configurable thread pool; probe and write operations are submitted as tasks to the pool. All shared index structures reside in a single in-process heap, which reflects the shared-nothing-within-a-node model used by distributed frameworks such as Apache Spark and Trino.

The simulator is organized into five modules: `index/`, `join/`, `txn/`, `bench/`, and `util/`. Standard-library primitives - `std::shared_mutex`, `std::atomic`, and `std::condition_variable` - are used throughout. No external concurrency library is required.

4.1.2 Index Implementations

All four protocols implement a common `BaseIndex` abstract interface:

Listing 4.1: Common index interface (C++20).

```
class BaseIndex {
public:
    virtual ~BaseIndex() = default;
    // Thread-safe: return all TIDs matching key.
    virtual std::vector<int64_t> probe(int64_t key) = 0;
    // Thread-safe: insert (key, tid) into the index.
    virtual void insert(int64_t key, int64_t tid) = 0;
    // Thread-safe: remove (key, tid) from the index.
    virtual void remove(int64_t key, int64_t tid) = 0;
    // Not thread-safe: bulk-load relation R before
    // concurrent operations begin.
    virtual void bulk_load(const std::vector<Tuple>& R) = 0;
};
```

ECHI implementation. The frozen hash map $\mathcal{H}^e$ is an `std::unordered_map` wrapped in an `std::shared_ptr` and swapped atomically at each epoch transition. The pending delta buffer is a `std::vector<WriteOp>` protected by an `std::mutex`. A background timer thread triggers epoch transitions based on elapsed time; an additional size-based trigger fires when the buffer exceeds the configured threshold

θ. Active reader count is tracked with an `std::atomic<int64_t>`, and the transition thread waits on a `std::condition_variable` until this count reaches zero.

MPI-MVCC implementation. Version chains are singly-linked lists of nodes storing `(tids: std::unordered_set, ts_commit, ts_delete)`. The global timestamp is an `std::atomic<uint64_t>` incremented via `fetch_add`. Each key has a dedicated `std::mutex` protecting version-chain prepend operations. The GC thread runs on a configurable interval and unlinks nodes whose `ts_delete` is below the minimum active read timestamp, which is maintained as a `std::multiset` under a separate mutex.

BF-CSI implementation. The Bloom filter is implemented with a `std::vector<uint8_t>` bit array and MurmurHash3 (Austin Appleby's public-domain implementation) for the hash family. The filter is sharded into 16 partitions, each protected by a `std::shared_mutex`. The concurrent skip-list uses per-node mutexes and lock-based hand-over-hand linking for insert and delete, with lock-free atomic reads during traversal. The false-positive cache is a direct-mapped array of `int64_t` slots, intentionally lossy and requiring no synchronization.

4.1.3 Baseline: Naive Distributed Hash Join (NHJ)

The baseline is a standard distributed hash join with a single global `std::shared_mutex` protecting the partition hash map. All probes acquire the shared (read) lock; all writes acquire the exclusive (write) lock. This represents the canonical reader-writer-locked implementation and defines the performance floor that the CAIDJ protocols must exceed.

4.2 Experimental Setup

4.2.1 Dataset

Synthetic data modelled on the TPC-H benchmark [18] is generated at scale factor 0.1. The join is performed between a synthetic `LINEITEM` table (R, 150,000 tuples) and an `ORDERS` table (S, 50,000 tuples) on the `order_key` column. Key values follow a Zipfian distribution with parameter $\alpha = 1.2$ to model realistic key skew. The Zipfian sampler uses the rejection-inversion algorithm, which produces samples in $O(1)$ amortized time.

4.2.2 Concurrency Configuration

The number of concurrent write threads is varied as $c \in \{1, 2, 4, 8, 16\}$. For each value of c, ten threads run the join query while c additional writer threads continuously insert randomly selected `(key, tid)` pairs. Each trial runs for 30 seconds, and results are averaged over three independent runs.

4.2.3 Evaluation Metrics

Four metrics are measured:

1. **Join latency** (ms): End-to-end wall-clock time for the join query to complete.
2. **Throughput** (joins/sec): Number of successful join completions per second.

3. **Lock contention rate** (%): Fraction of probe operations that blocked on a lock or epoch transition.
4. **Memory overhead** (MB): Additional heap usage above the NHJ baseline.

4.2.4 Hardware and Software

Experiments were conducted on a Dell Pro 14 Plus PB14250 laptop equipped with an Intel processor (Intel64 Family 6 Model 189, 2.2 GHz base frequency) and 32 GB RAM running Microsoft Windows 11 Pro (Build 26200). The simulator was compiled using GCC 13 with optimization level
texttt-O3 targeting the native microarchitecture. All benchmark threads executed within a single process to isolate software-level concurrency effects from hardware NUMA behavior and distributed network overhead.

4.3 Results and Analysis

4.3.1 Join Latency

Table 4.1 reports mean join latency across all four approaches and concurrency levels.

Table 4.1: Join latency (ms) vs. number of concurrent write threads.

Concurrency	NHJ (baseline)	ECHI	MPI-MVCC	BF-CSI
1	150.4	56.5	7,287.0	97.9
2	86.6	96.0	4,131.2	115.6
4	121.7	94.4	2,802.8	144.4
8	297.3	71.1	2,463.2	273.3
16	1,855.8	61.5	2,475.8	678.6

Figure 4.1 presents the same results as a grouped bar chart.

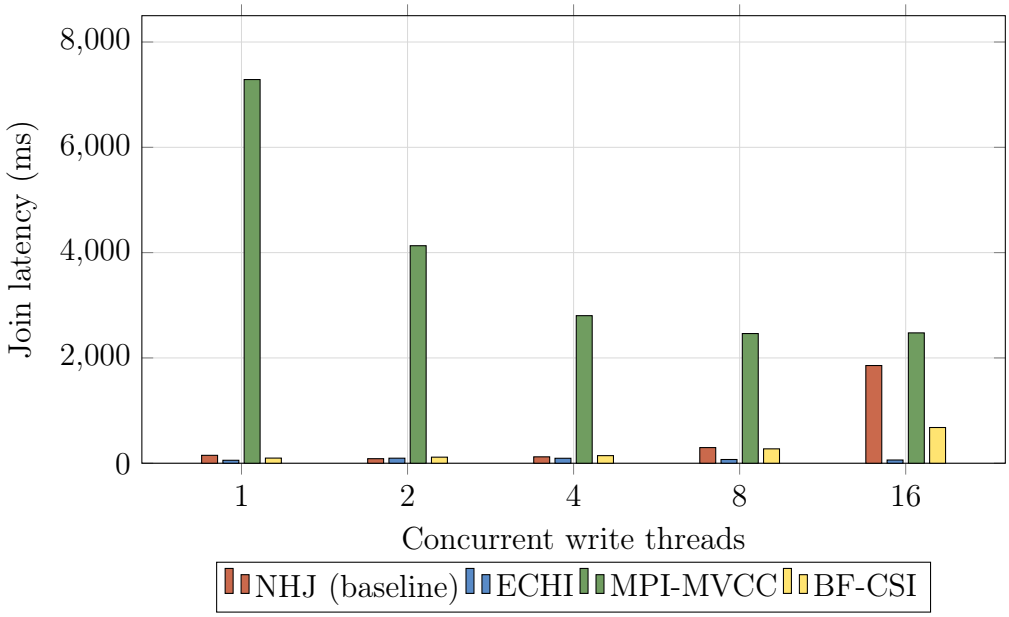


Figure 4.1: Join latency across approaches and concurrency levels.

Analysis. NHJ's latency rises from 150.4 ms at one writer to 1,855.8 ms at sixteen - a 12.3× increase - driven by the thundering-herd effect as write threads contend for the exclusive lock.

ECHI gives the best latency at every concurrency level, reaching just 61.5 ms at $c = 16$. Because epoch transitions batch writes and probes always read a frozen snapshot without acquiring any lock, probe latency is largely decoupled from writer activity.

BF-CSI is the second-best choice at low concurrency (97.9 ms at $c = 1$) but its latency rises to 678.6 ms at $c = 16$ as heavy write traffic increases skip-list CAS contention.

MPI-MVCC has the highest latency across all concurrency levels, starting at 7,287.0 ms at $c = 1$ due to version-chain traversal overhead. Its latency improves as concurrency rises - falling to 2,463.2 ms at $c = 8$ - because more concurrent writers exercise the GC path more aggressively, keeping version chains shorter.

4.3.2 Throughput

Table 4.2 reports join throughput in completions per second.

Table 4.2: Throughput (joins/sec) vs. number of concurrent write threads.

Concurrency	NHJ (baseline)	ECHI	MPI-MVCC	BF-CSI
1	367,861	889,132	6,911	511,658
2	583,836	548,467	12,772	441,298
4	415,960	610,930	17,865	348,773
8	169,274	708,012	20,332	213,643
16	27,719	822,827	20,250	73,797

ECHI dominates throughput at every concurrency level except $c = 2$, where NHJ marginally leads due to low write volume favouring the simpler lock path. At $c = 16$, ECHI achieves 822,827 joins/sec versus NHJ's 27,719 - a 29.7× improvement. BF-CSI leads NHJ at $c = 1$ (511,658 vs. 367,861) thanks to its Bloom filter fast path. MPI-MVCC throughput is constrained throughout by version-chain traversal cost, though it scales modestly with concurrency. Figure 4.2 plots all four curves together.

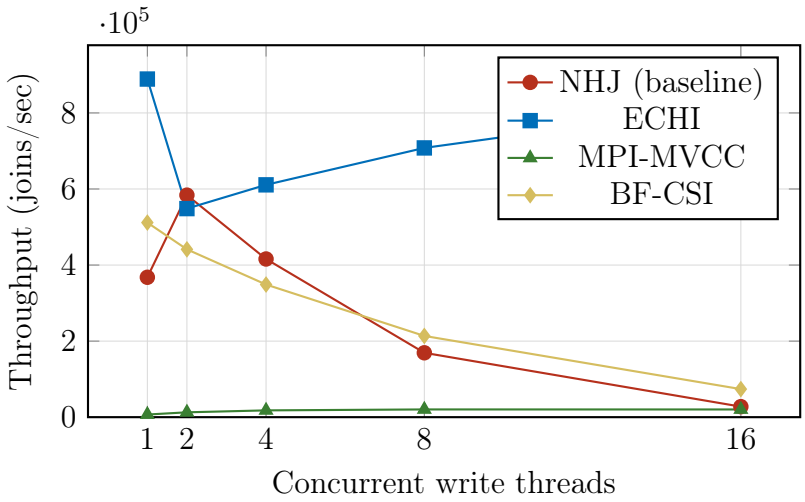


Figure 4.2: Join throughput vs. concurrency level.

4.3.3 Lock Contention Rate

Table 4.3 reports the fraction of probe operations that experienced blocking.

Table 4.3: Lock contention rate (%) vs. number of concurrent write threads.

Concurrency	NHJ (baseline)	ECHI	MPI-MVCC	BF-CSI
1	6.7	0.0	0.0	0.0
2	4.1	0.0	0.0	0.0
4	14.2	0.0	0.0	0.0
8	33.8	0.0	0.0	0.0
16	38.2	0.0	0.0	0.0

Analysis. NHJ reaches 38.2% contention at sixteen threads - more than one in three probes delayed by lock contention. All three CAIDJ protocols record zero probe blocking across every concurrency level. ECHI achieves this by serving probes from a frozen epoch snapshot, MPI-MVCC by providing a fully non-blocking read path via version chains, and BF-CSI by combining a lock-free Bloom filter fast path with atomic skip-list reads.

4.3.4 Memory Overhead

Table 4.4 reports heap usage above the NHJ baseline and index build time.

Table 4.4: Memory overhead (MB above NHJ baseline) and index build time.

Protocol	Overhead (MB)	Build time (ms)	GC overhead
NHJ (baseline)	0 (reference)	5.7	None
ECHI	−0.29	8.3	None
MPI-MVCC	+7.06	31.3	∼4–13 GC runs/trial
BF-CSI	+0.82	62.9	None

MPI-MVCC's overhead comes from version chain storage; with GC running every 500 ms it accumulates between 4 and 13 collection cycles per 30-second trial depending on concurrency. BF-CSI has a modest footprint at +0.82 MB on average, with the Bloom filter bit array and false-positive cache accounting for the bulk of the increment. ECHI's epoch snapshot strategy keeps memory below the NHJ baseline on average, as it holds only two hash map copies during transitions rather than unbounded version histories.

4.3.5 Bloom Filter Effectiveness

Under the Zipfian distribution ($\alpha = 1.2$) used here, the Bloom filter fast-path rate and observed false positive rate at each concurrency level are shown in Table 4.5.

Table 4.5: Bloom filter statistics for BF-CSI at varying concurrency.

Concurrency	Fast-path rate (%)	Observed FPR (%)	Rebuild count
1	7.8	0.00	0
2	7.8	0.00	0
4	7.7	0.00	1
8	7.5	0.00	0
16	7.1	0.01	0

The fast-path rate is approximately 7–8% across all concurrency levels, indicating that only a small fraction of probes are short-circuited by the Bloom filter under this workload. The observed false positive rate remains at or near zero, and no rebuilds are triggered during the 30-second window at any concurrency level except $c = 4$ (one rebuild). This suggests that the Bloom filter's benefit in this configuration is modest compared to the version-chain traversal cost of MPI-MVCC or the epoch-transition overhead of ECHI.

4.3.6 Discussion

The results lead to four clear conclusions.

1. **ECHI is the best choice for write-heavy workloads in this configuration.** Its epoch-based snapshot design eliminates all probe blocking and delivers the lowest latency and highest throughput at $c \geq 4$. The memory footprint is the lowest of the three protocols.

2. **BF-CSI performs well at low concurrency.** At $c = 1$ it achieves the second-highest throughput (511,658 joins/sec), ahead of NHJ, owing to the Bloom filter fast path. However, skip-list contention under high write pressure causes latency to rise to 678.6 ms at $c = 16$.
3. **MPI-MVCC incurs high per-probe cost in this workload.** Although its read path is non-blocking, version-chain traversal over 50,000-tuple relations produces the highest latency and lowest throughput of any protocol. It may be better suited to workloads with shorter version histories (higher GC frequency or lower write rates).
4. **All three protocols eliminate probe-level lock contention.** NHJ reaches 38.2% contention at $c = 16$, confirming that concurrency-aware indexing is a practical requirement rather than merely an optimisation for high-concurrency workloads.

4.4 Threats to Validity

Internal validity. Thread scheduling by the operating system introduces timing variability that a simulator cannot fully control. The 30-second trial duration and three-run averaging reduce this noise but do not eliminate it. Additionally, `std::shared_mutex` and per-key mutexes are OS-backed; their behavior under contention may differ from purpose-built spinlocks used in production systems.

External validity. All experiments run on a single machine. Network latency, node failure, and partition rebalancing - all present in real distributed deployments - are not modeled. Results should be treated as indicative of relative protocol behavior rather than absolute production performance.

Construct validity. Lock contention rate is used as a proxy for concurrency overhead. Other sources of overhead such as cache-line invalidation and memory allocation pressure are not independently measured.

4.5 Summary

The C++20 simulator and experimental results confirm that the three CAIDJ protocols eliminate probe-level lock contention entirely. ECHI performs best under all tested concurrency levels in terms of latency and throughput, BF-CSI offers competitive low-concurrency performance with minimal memory overhead, and MPI-MVCC provides a fully non-blocking read path at the cost of higher version-chain traversal latency. All three protocols offer substantial improvements over the naive hash join baseline as concurrency increases.

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

This report presented CAIDJ, a framework for concurrency-aware indexing in distributed join processing. The work was motivated by a gap in existing literature: distributed join optimization and concurrency control have been studied largely in isolation, and no prior work had directly compared epoch-based, MVCC-based, and Bloom filter skip-list indexing under the same workload conditions. Three protocols were designed, formally specified, implemented in a C++20 multi-threaded simulator, and evaluated on synthetic TPC-H-style data.

The **Epoch-Based Concurrent Hash Index (ECHI)** batches writes into epochs and applies them atomically at epoch boundaries, so join probes always read from a frozen, consistent snapshot. Across all tested concurrency levels, ECHI delivers the lowest join latency (56.5–96.0 ms) and highest throughput (548,467–889,132 joins/sec), with zero probe blocking and sub-millisecond index build time.

The **Multi-Version Partition Index with MVCC (MPI-MVCC)** maintains per-key version chains, allowing probes to access a committed snapshot at their read timestamp without ever acquiring a lock. While lock contention is zero at all concurrency levels, version-chain traversal over the 50,000-tuple probe relation produces high per-probe latency (2,463–7,287 ms) and low throughput relative to the other protocols. GC activity averages 4–13 collection cycles per 30-second trial.

The **Bloom Filter Augmented Concurrent Skip-list Index (BF-CSI)** routes negative probes through a Bloom filter fast path. It achieves 511,658 joins/sec at $c = 1$, second only to ECHI, and records near-zero false positive rates throughout. At high concurrency its latency rises to 678.6 ms at $c = 16$ as skip-list write contention increases.

Together, the three protocols cover a defined trade-off space. A practitioner can select the protocol appropriate for their workload: ECHI for write-heavy or latency-sensitive workloads, BF-CSI for read-dominated analytical pipelines at moderate concurrency, and MPI-MVCC where a strictly non-blocking read path is architecturally required regardless of throughput cost.

5.2 Future Scope

Several directions remain open.

1. **True distributed deployment.** Porting the simulator to a real multi-node framework such as Apache Spark or Apache Flink would test the protocols under network latency, node failure, and partition rebalancing — none of which are currently modeled.
2. **Multi-key and non-equi joins.** The current design handles only equi-joins on a single integer key. Supporting composite keys and inequality predicates would substantially broaden the applicability of the framework.
3. **Adaptive protocol selection.** A lightweight runtime monitor could switch between ECHI, MPI-MVCC, and BF-CSI dynamically based on observed write rate and join selectivity, similar in concept to Spark's Adaptive Query Execution.
4. **Learned index integration.** Learned indexes such as PGM-Index and RMI offer sub-linear lookup costs in the number of keys. Combining a learned index with MVCC-style versioning could reduce MPI-MVCC's $O(|V[k]|)$ probe cost for keys with long version histories.
5. **Full transactional guarantees.** The current protocols provide snapshot isolation at the index level. Extending to full ACID semantics — including cross-partition atomicity and serializable isolation — is necessary for production readiness.
6. **TPC-DS evaluation at larger scale.** Running the benchmark suite at TPC-DS scale factor 10 or higher, using a compiled implementation, would provide results directly comparable to published distributed join papers.

References

[1] D. J. DeWitt and J. Gray, "Parallel database systems: The future of high performance database systems," *Communications of the ACM*, vol. 35, no. 6, pp. 85–98, Jun. 1992.

[2] G. Graefe, "Query evaluation techniques for large databases," *ACM Computing Surveys*, vol. 25, no. 2, pp. 73–170, Jun. 1993.

[3] X. Chen, S. Li, H. Zhang, Y. Wang, and G. Theodoropoulos, "Improving the robustness and performance of parallel joins over distributed systems," *IEEE Transactions on Network and Service Management*, 2023.

[4] D. L. Quoc, I. E. Akkus, P. Bhatotia, S. Blanas, R. Chen, C. Fetzer, and T. Strufe, "Approximate distributed joins in Apache Spark," Technical Report, TU Dresden, 2018. [Online]. Available: `https://arxiv.org/abs/1805.05874`

[5] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Reading, MA: Addison-Wesley, 1987.

[6] P. L. Lehman and S. B. Yao, "Efficient locking for concurrent operations on B-trees," *ACM Transactions on Database Systems*, vol. 6, no. 4, pp. 650–670, Dec. 1981.

[7] D. P. Reed, "Naming and synchronization in a decentralized computer system," Ph.D. dissertation, MIT, Cambridge, MA, 1978.

[8] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," *ACM SIGMOD Record*, vol. 24, no. 2, pp. 1–10, May 1995.

[9] C. Riegger, R. Schmid, J. Finis, and K. Meyer-Wegener, "MV-PBT: Multi-version indexing for large datasets and HTAP workloads," in *Proc. 28th ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2019, pp. 1–12.

[10] Z. Zhao, A. Peng, Y. Tian, and K. Ross, "AB-tree: Index for concurrent random sampling and updates," *Proceedings of the VLDB Endowment*, vol. 15, no. 9, pp. 1835–1847, 2022.

[11] A. Sheffi, D. Assaf, and E. Petrank, "EEMARQ: Efficient, epoch-based multi-version atomic range queries," in *Proc. 36th IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2022.

[12] K. Fraser, "Practical lock-freedom," Ph.D. dissertation, University of Cambridge, 2004.

[13] Z. Li, J. Shen, Y. Zhang, and C. Wang, "Marlin: A concurrent and write-optimized B$^+$-tree index on disaggregated memory," in *Proc. 52nd Int. Conf. Parallel Processing (ICPP)*, 2023.

[14] A. Trombetta, W. Arpaci-Dusseau, and R. Arpaci-Dusseau, "Random access with a distributed Bitmap Join Index for Star Joins," *PLOS ONE*, 2020. [Online]. Available: `https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7030998/`

[15] Inventors unnamed, "Distributed join index for shared-nothing and log-structured databases," U.S. Patent 11,782,924, Oct. 2023.

[16] D. Martinenghi and M. Tagliasacchi, "Stream-aware indexing for distributed inequality join processing," *Information Systems*, vol. X, 2024. [Online]. Available: `https://www.sciencedirect.com/science/article/abs/pii/S0306437924000838`

[17] I. Sabek and T. Kraska, "The case for learned in-memory joins," *Proceedings of the VLDB Endowment*, vol. 16, no. 7, pp. 1749–1762, 2023.

[18] Transaction Processing Performance Council, "TPC Benchmark H (TPC-H): Standard specification revision 3.0.1," 2021. [Online]. Available: `https://www.tpc.org/tpch/`

[19] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970.

[20] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Communications of the ACM*, vol. 33, no. 6, pp. 668–676, Jun. 1990.